PROJECT REPORT

Emergency Ambulance Allocation System

Course Name & Code: UCS310 – Database Management Systems
Degree & Year: B.Tech – 2nd Year
Department: Computer Engineering
Institute: Thapar Institute of Engineering and Technology, Patiala

Group Members

Rabeeh Ahmad – 1024030716
Ishan Singhania – 1024030719
Tushar Sharma – 1024030676

1. INTRODUCTION

Emergency medical services (EMS) are among the most time-critical systems in public infrastructure. The difference of a few minutes in dispatching an ambulance to the scene of a cardiac arrest, road accident, or breathing emergency can mean the difference between life and death. Despite this urgency, ambulance dispatch in many regions continues to rely on manual processes: operators take calls, verbally communicate with drivers, and maintain paper records or isolated spreadsheets to track resource availability.

These file-based, manual systems suffer from well-documented limitations: there is no real-time view of ambulance locations, no automated mechanism to find the nearest available unit, no way to guarantee hospital bed availability before dispatching, and no protection against two dispatchers sending the same ambulance to two different emergencies simultaneously. The result is wasted response time, duplicated effort, and a serious risk of resource conflict.

The Emergency Ambulance Allocation System addresses these problems head-on. It is a full-stack, database-centric web application that automates the entire dispatch lifecycle — from the moment an emergency call is logged to the moment the ambulance completes its assignment and returns to service. Critically, all business logic — ambulance selection, hospital assignment, ACID-compliant resource locking, and completion handling — lives entirely inside the MySQL database layer through PL/SQL stored procedures, functions, and triggers. The application backend (Node.js + Express) serves purely as a thin REST API bridge, while a single-page HTML/CSS/JavaScript dispatcher dashboard provides the user interface.

This project was selected because it represents a genuine, high-stakes real-world problem whose correct solution demands every major DBMS concept: relational schema design, normalization, SQL joins and aggregates, PL/SQL stored procedures, user-defined functions, triggers, views, transaction management, and concurrency control. It is not a toy application — the data model and procedure logic are designed to the same standards one would apply in a production emergency response system.

Why a Database Management System Instead of a File-Based System

Concurrent access and data integrity: A file-based system has no built-in concurrency control. If two dispatchers simultaneously open and edit a flat file, both may read the same ambulance as “available” and dispatch it to different emergencies — a dangerous double-booking. MySQL’s InnoDB engine with SELECT ... FOR UPDATE row-level locking completely eliminates this race condition.

ACID transactions: Allocating an ambulance involves four separate write operations (INSERT into Assignment, UPDATE Ambulance status, DECREMENT hospital bed count, UPDATE request status). In a flat file, a crash mid-way leaves the data inconsistent. A MySQL transaction wraps all four into a single atomic unit — either all succeed or all roll back.

Structured querying: Finding the geographically nearest available ambulance requires computing distances across all available units and sorting them. This is trivially expressed as an ORDER BY clause over a MySQL scalar function. In a flat file, this requires custom application code for every such query.

Referential integrity: Foreign key constraints ensure that an Assignment can never reference a non-existent ambulance or hospital. Flat files have no such enforcement.

Triggers for automation: MySQL triggers automatically handle side effects — freeing an ambulance when a job is marked complete, guarding against bed-count violations — without any application code. Flat files offer no equivalent mechanism.

2. PROBLEM STATEMENT

Emergency medical dispatch systems that rely on manual coordination and file-based records are fundamentally incapable of meeting the speed and reliability demands of modern emergency response. The following specific problems are addressed by this project:

Real-World Problem Definition

- **No real-time resource visibility:** Dispatchers have no live view of which ambulances are available, where they are, or which hospitals have beds. Decisions are made on stale information.
- **No automated nearest-unit selection:** Selecting the nearest available ambulance manually is slow and error-prone. A dispatcher may send a far unit simply because they happen to know its number.
- **No hospital bed verification:** An ambulance may arrive at a hospital only to discover no beds are available, forcing a secondary transfer — a dangerous and costly delay.
- **Race conditions in concurrent dispatch:** When multiple dispatchers operate simultaneously, two may read the same ambulance as available and assign it to different calls, creating a double-booking conflict with no resolution mechanism.
- **No structured audit trail:** Flat files and paper records cannot provide a reliable history of who was dispatched where, when, and for how long. Reporting and accountability are impossible.

Limitations of the Existing Manual System

- Ambulance status is tracked on whiteboards or spreadsheets with no automatic update when a unit is dispatched or returns.
- Hospital bed counts are communicated verbally or by telephone, with no guaranteed consistency.
- There is no enforced policy to prevent a completed request from being re-assigned.
- There is no mechanism to detect or prevent concurrent conflicting assignments.
- Generating statistics — peak hours, hospital load, per-ambulance utilization — requires manual compilation.

Scope of Database Usage in Solving the Problem

The database is the system. Every decision — which ambulance to send, which hospital to assign, how to ensure no double-booking — is made inside stored procedures executing within a single ACID transaction. The application layer

merely triggers these procedures and renders the results. This architecture guarantees that the data is always consistent regardless of how many simultaneous users or API calls are active.

3. OBJECTIVES OF THE PROJECT

1. To design a fully normalized relational database schema using Entity–Relationship (ER) data modelling, identifying all entities, attributes, relationships, and cardinalities relevant to emergency ambulance dispatch.
2. To convert the ER model into a relational schema in Third Normal Form (3NF), eliminating all partial and transitive dependencies.
3. **To implement all database structures in MySQL** using DDL commands (CREATE TABLE with appropriate data types, constraints, and foreign keys) and populate them with realistic seed data using DML commands.
4. To write complex SQL SELECT queries using multi-table JOINs, aggregate functions (COUNT, SUM, ROUND), GROUP BY, HAVING, and subqueries to power the reporting and dashboard features of the application.
5. **To implement a PL/SQL stored function** (calculate_distance) that encapsulates the Haversine great-circle distance formula, enabling the database itself to rank ambulances by proximity to an emergency.
6. **To implement a core PL/SQL stored procedure** (allocate_ambulance) that atomically selects the nearest available ambulance, assigns a hospital with available beds, updates all resource statuses, and commits or rolls back the entire operation as a single ACID transaction.
7. To implement three database triggers: a BEFORE INSERT trigger to guard against assigning a full hospital, an AFTER INSERT trigger as a safety net for direct inserts, and an AFTER UPDATE trigger to automatically free ambulance resources upon job completion.
8. To create three SQL views (available_ambulances, active_assignments, hospital_capacity) to simplify complex JOIN queries used repeatedly by the dashboard.
9. To demonstrate and document concurrency control through SELECT ... FOR UPDATE row-level locking, preventing double-booking under simultaneous dispatch scenarios.
10. To ensure all four ACID properties — Atomicity, Consistency, Isolation, and Durability — are explicitly satisfied and documented with reference to specific database constructs.
11. To build a complete full-stack application (Node.js REST API + single-page HTML/CSS/JS dashboard) that demonstrates the database system in a real operational context.

4. SCOPE OF THE PROJECT

Functional Boundaries

The system manages the complete lifecycle of an emergency dispatch event, from call intake to ambulance return-to-service. It does not include patient medical records, billing, or vehicle GPS tracking (GPS coordinates are stored as static attributes and would be updated by an external telematics feed in a production deployment).

Types of Users

- **Dispatcher (primary user):** Logs incoming emergency calls, triggers allocation, monitors the live dashboard, and marks jobs as complete.
- **Hospital administrator:** Updates available bed counts through the Hospitals management page when patients are discharged or new capacity becomes available.
- **System administrator:** Manages the database, seeds driver and ambulance records, and accesses the Reports page for operational analytics.

Modules Covered

- **Emergency Request Management:** Create, log, and auto-allocate incoming calls. Manual re-allocation for requests that could not be assigned initially.
- **Ambulance Fleet Management:** Real-time view of all units, their GPS coordinates, driver assignments, and status. Mark-done functionality that triggers the completion chain.
- **Hospital Capacity Management:** Live bed occupancy tracking, visual capacity bars, and manual bed-count updates.
- **Assignment Tracking:** Full join view of every allocation ever made, filterable by active or completed, with completion action for active rows.
- **Reporting and Analytics:** Daily summary (PENDING / ASSIGNED / COMPLETED counts), assignments per hospital, top ambulances by volume, and hourly request distribution.
- **Database Core:** All five tables, three views, one function, one stored procedure, three triggers, and all supporting SQL queries.

5. PROPOSED SYSTEM DESCRIPTION

The Emergency Ambulance Allocation System is a three-tier architecture where the database tier is the most significant layer. The system operates as follows:

A dispatcher opens the web dashboard and logs an incoming emergency call by filling a form with the caller's name, phone number, emergency type (Cardiac arrest, Road accident, Breathing difficulty, Fall injury, or Other), priority level (Critical, High, or Low), and the GPS latitude/longitude of the emergency location. Upon submission, the frontend sends a POST request to the Node.js backend.

The backend inserts a new row into Emergency_Request with status = PENDING and immediately calls the allocate_ambulance stored procedure, passing the new request ID. Inside the procedure, a transaction begins. The procedure reads the emergency coordinates, then queries the Ambulance table to find all units with status = AVAILABLE, computes the great-circle distance from each unit to the emergency using the Haversine function, and selects the nearest one with a FOR UPDATE lock. It simultaneously selects the first hospital with available_beds > 0 with a FOR UPDATE lock. If either resource is absent, the transaction rolls back and the request stays PENDING. If both are found, the procedure inserts a row into Assignment, sets the ambulance to BUSY, decrements the hospital's available_beds by one, sets the request to ASSIGNED, and commits.

The dashboard displays live metrics: count of PENDING requests (amber), count of BUSY ambulances (blue), count of AVAILABLE ambulances (green), and count of COMPLETED assignments today. It also shows a recent-requests list, hospital occupancy bars, and an ambulance fleet strip, all auto-refreshed every 30 seconds.

When the ambulance completes its assignment, the dispatcher clicks "Mark done" on either the Ambulances or Assignments page. The backend sets completion_time = NOW() on the Assignment row. The AFTER UPDATE trigger fires automatically: it sets the ambulance back to AVAILABLE and the request to COMPLETED. No additional API calls are required.

Key Features and Functionalities

- GPS-based nearest-ambulance selection using the Haversine formula implemented as a MySQL scalar function
- ACID-compliant allocation stored procedure with full transaction management and exception handling
- Row-level locking (SELECT ... FOR UPDATE) preventing concurrent double-booking

- Three database triggers enforcing data integrity throughout the assignment lifecycle
- Three SQL views simplifying complex multi-table joins for dashboard queries
- 14 REST API endpoints covering full CRUD for all five entities
- Single-page dashboard with six functional pages: Dashboard, Emergency Requests, Ambulances, Hospitals, Assignments, and Reports
- Real-time auto-refresh every 30 seconds
- Manual bed-count management for hospital administrators

6. DATABASE DESIGN

6.1 Entity–Relationship (ER) Diagram

The following entities, attributes, relationships, and cardinalities were identified during the design phase:

Entities and Attributes

Entity	Primary Key	Key Attributes
Driver	driver_id	name, phone (UNIQUE), license_number (UNIQUE), experience_years, status
Ambulance	ambulance_id	vehicle_number (UNIQUE), status (ENUM), current_latitude, current_longitude
Hospital	hospital_id	name, address, contact_number, total_beds, available_beds (CHECK >= 0)
Emergency_Request	request_id	caller_name, caller_phone, request_time, emergency_type, location coordinates, priority (ENUM), status (ENUM)
Assignment	assignment_id	request_id (UNIQUE FK), ambulance_id (FK), hospital_id (FK), assigned_time, completion_time

Relationships and Cardinalities

Relationship	Cardinality	Description
Driver – Ambulance	1 : N	One driver may be assigned to one or more ambulances; each ambulance has at most one driver
Emergency_Request – Assignment	1 : 1	One request produces exactly one assignment (UNIQUE constraint on request_id in Assignment)
Ambulance – Assignment	1 : N	One ambulance can have many assignments over time but only

Relationship	Cardinality	Description
		one active (non-completed) at a time
Hospital – Assignment	1 : N	One hospital can receive many assignments across time; bed count is decremented per active assignment

Note: The ER diagram (graphical form) is to be attached as Appendix A in the final report.

6.2 Relational Schema

Driver

driver_id INT PK AUTO_INCREMENT, name VARCHAR(100) NOT NULL, phone VARCHAR(15) UNIQUE NOT NULL, license_number VARCHAR(50) UNIQUE NOT NULL, experience_years INT, status VARCHAR(20)

Ambulance

ambulance_id INT PK AUTO_INCREMENT, vehicle_number VARCHAR(20) UNIQUE NOT NULL, driver_id INT FK → Driver(driver_id), status ENUM('AVAILABLE','BUSY') NOT NULL DEFAULT 'AVAILABLE', current_latitude DOUBLE, current_longitude DOUBLE

Hospital

hospital_id INT PK AUTO_INCREMENT, name VARCHAR(150) NOT NULL, address TEXT, contact_number VARCHAR(15), total_beds INT NOT NULL, available_beds INT NOT NULL CHECK(available_beds >= 0)

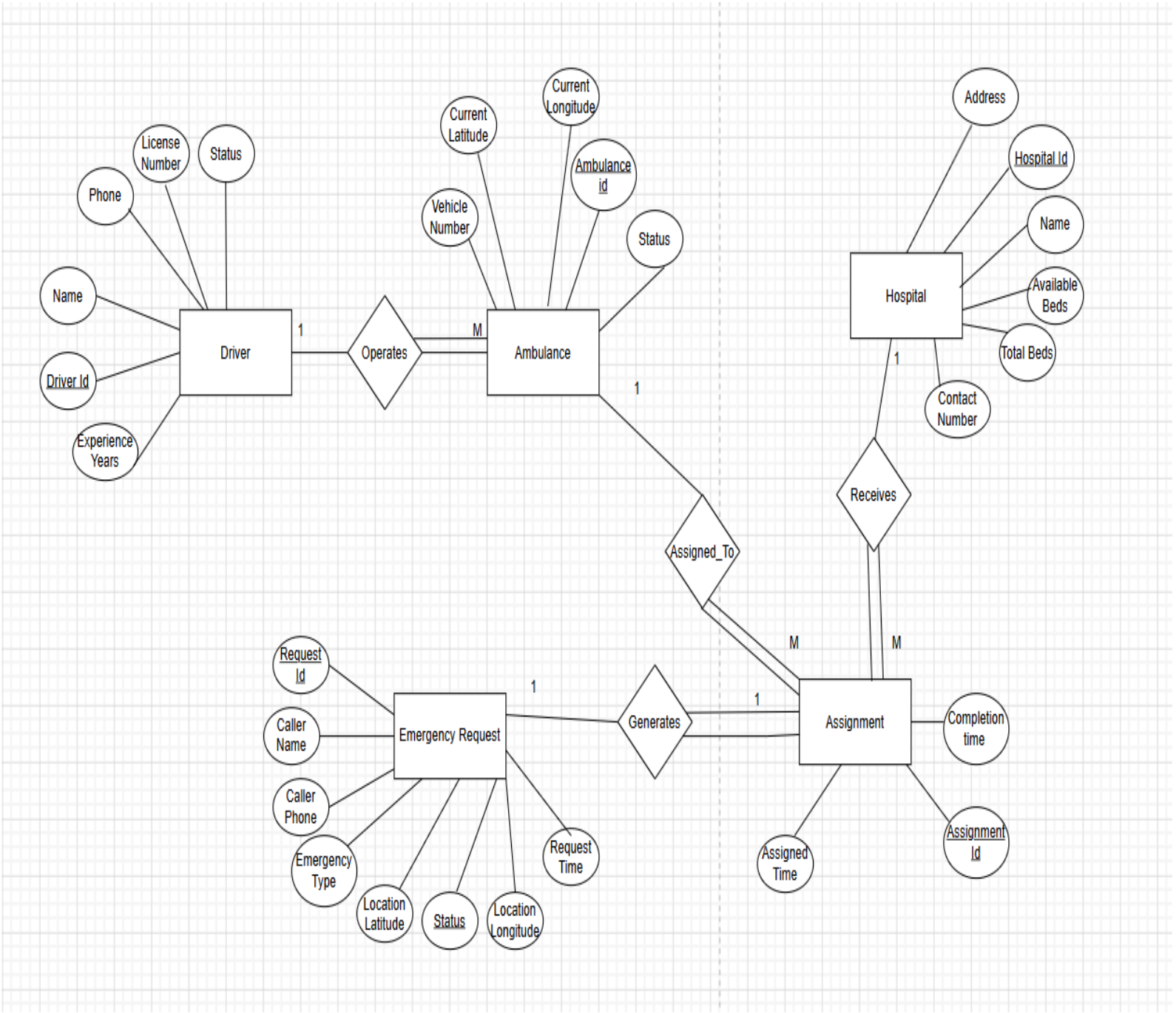
Emergency_Request

request_id INT PK AUTO_INCREMENT, caller_name VARCHAR(100), caller_phone VARCHAR(15), request_time DATETIME DEFAULT CURRENT_TIMESTAMP, emergency_type VARCHAR(100), location_latitude DOUBLE, location_longitude DOUBLE, priority ENUM('Critical','High','Low') NOT NULL, status ENUM('PENDING','ASSIGNED','COMPLETED') NOT NULL DEFAULT 'PENDING'

Assignment

assignment_id INT PK AUTO_INCREMENT, request_id INT UNIQUE NOT NULL FK → Emergency_Request(request_id), ambulance_id INT NOT NULL FK → Ambulance(ambulance_id), hospital_id INT NOT NULL FK →

Hospital(hospital_id), assigned_time DATETIME DEFAULT CURRENT_TIMESTAMP, completion_time DATETIME



7. NORMALIZATION

7.1 First Normal Form (1NF)

All tables satisfy 1NF. Every attribute holds only atomic (indivisible) values — there are no repeating groups or multi-valued attributes. Each table has a single-column integer primary key. For example, rather than storing multiple phone numbers in one field, each driver has exactly one phone (enforced with UNIQUE NOT NULL). GPS coordinates are stored as two separate DOUBLE columns (current_latitude, current_longitude) rather than a single composite string.

7.2 Second Normal Form (2NF)

All tables satisfy 2NF. Since every table uses a single-column primary key (no composite keys), partial dependency cannot exist by definition. Every non-key attribute in each table is fully functionally dependent on its entire primary key. In the Assignment table for example, assigned_time and completion_time depend on assignment_id, not on any proper subset of it.

7.3 Third Normal Form (3NF)

All tables satisfy 3NF. No non-key attribute depends on another non-key attribute (no transitive dependencies). The design choices that enforce this include:

- **Driver details not stored in Ambulance:** The Ambulance table stores only driver_id as a foreign key. Driver name, phone, and license are stored exclusively in the Driver table, eliminating the transitive dependency driver_id → driver_name inside Ambulance.
- **Hospital details not stored in Assignment:** Assignment stores only hospital_id. Hospital name, address, and bed counts reside in the Hospital table, preventing transitive dependencies through hospital_id.
- **Request details not duplicated in Assignment:** Assignment stores only request_id. Caller name, emergency type, and coordinates remain in Emergency_Request, with no duplication.
- **No derived attributes stored:** The used bed count is always computed as total_beds − available_beds at query time; it is never stored as a separate column, preventing potential update anomalies.

Table	1NF	2NF	3NF	Reason
Driver	✓	✓	✓	Single PK; all attributes

Table	1NF	2NF	3NF	Reason
				depend only on driver_id
Ambulance	✓	✓	✓	driver_id is FK, no driver details stored here
Hospital	✓	✓	✓	No transitive deps; bed count is atomic
Emergency_Request	✓	✓	✓	All attributes depend solely on request_id
Assignment	✓	✓	✓	Pure join table; no non-key transitive deps

8. DATABASE IMPLEMENTATION

8.1 SQL Implementation

DDL — Table Creation with Constraints

All five tables are created with explicit data type choices, NOT NULL constraints, UNIQUE constraints, ENUM domains for status fields, and a CHECK constraint on Hospital.available_beds ≥ 0. Foreign key relationships are declared with REFERENCES clauses, enabling InnoDB to enforce referential integrity at the engine level. The full CREATE TABLE statements are provided in Appendix B.

DML — Data Manipulation

The system uses parameterized INSERT statements (via the Node.js mysql2 library) to insert new Emergency_Request rows safely, preventing SQL injection. UPDATE statements are used by the stored procedure to change ambulance status and hospital bed counts. The Node.js backend also exposes a PUT endpoint for manual bed-count updates by hospital administrators.

SELECT Queries — Joins

Full assignment detail query (5-table join):

This query joins Assignment with Emergency_Request, Ambulance, Driver, and Hospital to produce a complete dispatch record for the Assignments dashboard page. It demonstrates INNER JOIN across all five tables, retrieving caller details, vehicle number, driver name, hospital name, assigned time, and completion time in a single query.

Available ambulances with driver info:

A JOIN between Ambulance and Driver filtered by status = 'AVAILABLE', used as the basis for the available_ambulances view and the dashboard fleet strip.

SELECT Queries — Aggregates and GROUP BY

Assignments per hospital:

SELECT h.name, COUNT(a.assignment_id) AS total_assignments FROM Hospital h LEFT JOIN Assignment a ON h.hospital_id = a.hospital_id GROUP BY h.hospital_id, h.name ORDER BY total_assignments DESC. This powers the Reports page bar chart and demonstrates LEFT JOIN to include hospitals with zero assignments, COUNT aggregation, GROUP BY, and ORDER BY.

Daily summary:

Uses conditional aggregation (SUM with CASE WHEN) to count PENDING, ASSIGNED, and COMPLETED requests for the current date in a single pass over the Emergency_Request table, avoiding three separate queries.

Top ambulances by assignment volume:

Joins Ambulance with Driver and Assignment, groups by ambulance, and orders by COUNT(assignment_id) DESC to identify the most-utilized units.

Pending requests over time (hourly distribution):

Uses HOUR(request_time) with GROUP BY to bin requests into hourly slots for the current date, enabling identification of peak demand periods.

Views

View Name	Purpose
available_ambulances	JOIN between Ambulance and Driver filtered to status = AVAILABLE. Provides a clean, named abstraction for dispatchers and dashboard queries.
active_assignments	5-table JOIN filtered to assignments with completion_time IS NULL. Powers the Assignments page Active filter tab and the dashboard active-count metric.
hospital_capacity	Selects hospital_id, name, total_beds, available_beds, and computes availability_percent using ROUND((available_beds / total_beds) * 100, 1). Powers the hospital capacity bars on the dashboard.

8.2 PL/SQL Components

Stored Function — calculate_distance

The calculate_distance function accepts four DOUBLE parameters (lat1, lon1, lat2, lon2) and returns the great-circle distance in kilometres between two GPS coordinate pairs using the Haversine formula:

distance = 6371 × ACOS(COS(RADIANS(lat1)) × COS(RADIANS(lat2)) × COS(RADIANS(lon2) − RADIANS(lon1)) + SIN(RADIANS(lat1)) × SIN(RADIANS(lat2)))

The function is declared DETERMINISTIC, informing the query optimizer that identical inputs always produce identical outputs, enabling caching and optimization. It is called inside the allocate_ambulance procedure as the ORDER BY expression when ranking available ambulances, meaning the database engine — not the application — selects the geographically nearest unit. A GREATEST/LEAST guard prevents floating-point domain errors in the ACOS call.

Stored Procedure — allocate_ambulance

This is the central business logic unit of the entire system. It accepts a single IN parameter p_request_id INT and executes the following sequence within a single transaction:

12. **Read coordinates:** SELECT location_latitude, location_longitude INTO local variables from Emergency_Request WHERE request_id = p_request_id.
13. **Lock nearest ambulance:** SELECT ambulance_id INTO v_ambulance_id FROM Ambulance WHERE status = 'AVAILABLE' ORDER BY calculate_distance(...) LIMIT 1 FOR UPDATE. The FOR UPDATE clause acquires a row-level exclusive lock on the selected ambulance row, preventing any concurrent transaction from modifying it.
14. **Lock hospital:** SELECT hospital_id INTO v_hospital_id FROM Hospital WHERE available_beds > 0 LIMIT 1 FOR UPDATE. Similarly locks the selected hospital row.
15. **Guard check:** IF v_ambulance_id IS NULL OR v_hospital_id IS NULL THEN ROLLBACK — request remains PENDING with no partial state.
16. **Commit allocation:** INSERT into Assignment, UPDATE Ambulance status to BUSY, DECREMENT hospital available_beds by 1, UPDATE Emergency_Request status to ASSIGNED, COMMIT.

An EXIT HANDLER FOR SQLEXCEPTION ensures that any unexpected SQL error also triggers a ROLLBACK, guaranteeing the database is never left in an inconsistent state.

Triggers

Trigger 1 — before_assignment_insert (BEFORE INSERT on Assignment):

Reads the available_beds count for the target hospital. If the count is ≤ 0, the trigger raises a SIGNAL SQLSTATE '45000' with message "No available beds in selected hospital", aborting the INSERT and rolling back the enclosing transaction. This acts as a final database-level guard against assigning a full hospital, independent of the application layer.

Trigger 2 — after_assignment_insert (AFTER INSERT on Assignment):

Updates Ambulance status to BUSY and decrements Hospital available_beds after a successful INSERT. In normal operation, these updates are already performed explicitly by the stored procedure. This trigger serves as a safety net for any INSERT executed directly against the Assignment table outside the procedure — ensuring data consistency regardless of how the insert originates.

Trigger 3 — after_assignment_update (AFTER UPDATE on Assignment):

Fires whenever an Assignment row is updated. It checks IF NEW.completion_time IS NOT NULL AND OLD.completion_time IS NULL — meaning a job has just been marked complete for the first time. When this condition holds, the trigger automatically sets Ambulance.status = 'AVAILABLE' for the previously busy ambulance and sets Emergency_Request.status = 'COMPLETED' for the linked request. This trigger-driven completion chain means the backend only needs to execute a single UPDATE statement; all downstream effects are handled by the database automatically.

Exception Handling

The allocate_ambulance procedure declares an EXIT HANDLER FOR SQLEXCEPTION that executes ROLLBACK. This means any SQL error during the procedure — a constraint violation, a deadlock, a connection loss — causes the entire transaction to be undone cleanly. The before_assignment_insert trigger uses SIGNAL to raise a custom exception that propagates up through the procedure's exception handler, guaranteeing consistent rollback behaviour at every level.

9. TRANSACTION MANAGEMENT AND CONCURRENCY CONTROL

9.1 Transaction Management

The allocate_ambulance stored procedure wraps its entire operation in an explicit transaction (START TRANSACTION ... COMMIT / ROLLBACK). This guarantees all four ACID properties:

ACID Property	How It Is Satisfied
Atomicity	All four write operations (INSERT Assignment, UPDATE Ambulance, UPDATE Hospital, UPDATE Emergency_Request) are inside a single transaction. If any step fails, ROLLBACK undoes all of them. There is no partially-committed allocation.
Consistency	CHECK constraint (available_beds >= 0), ENUM constraints on status fields, UNIQUE constraint on Assignment.request_id, foreign key constraints, and the BEFORE INSERT trigger collectively prevent the database from reaching any invalid state.
Isolation	SELECT ... FOR UPDATE acquires exclusive row-level locks on the selected ambulance and hospital. Concurrent transactions attempting to read the same rows in their own FOR UPDATE clauses will block until the first transaction commits or rolls back, preventing dirty reads and lost updates.
Durability	Once COMMIT executes successfully, the allocation is written to the InnoDB redo log and is permanent, surviving any subsequent crash or restart.

9.2 Concurrency Control — The Double-Booking Scenario

Consider two emergency calls arriving within milliseconds of each other, both handled by separate Node.js API threads:

17. Transaction T1 begins. It reads the emergency coordinates for request #10 and issues SELECT ambulance_id FROM Ambulance WHERE status

= AVAILABLE ORDER BY distance LIMIT 1 FOR UPDATE. InnoDB finds AMB-003 as the nearest unit and locks its row.

1. Transaction T2 begins. It reads coordinates for request #11 and issues the same query. InnoDB sees that AMB-003's row is already exclusively locked by T1. T2 blocks and waits.
2. T1 completes: it inserts the Assignment, sets AMB-003 to BUSY, decrements the hospital bed, and COMMITs. The lock on AMB-003 is released.
3. T2 resumes. It now re-evaluates the available ambulances and finds AMB-003 is BUSY. It selects the next nearest available unit, say AMB-007, locks it, and completes its own allocation. AMB-003 is never double-booked.

This sequence is guaranteed by MySQL's InnoDB engine. Without FOR UPDATE, both transactions could read AMB-003 as available simultaneously and both attempt to assign it, resulting in a double-booking. Row-level locking eliminates this race condition entirely.

10. TOOLS AND TECHNOLOGIES USED

Category	Technology	Purpose
DBMS	MySQL 8.0	Relational database engine; InnoDB storage for transactions and row-level locking
Database Language	SQL / PL/SQL (MySQL dialect)	DDL, DML, stored procedures, functions, triggers, views, cursors
Backend Runtime	Node.js v18+	JavaScript runtime for the REST API server
Backend Framework	Express.js 4.x	HTTP routing and middleware for the REST API
Database Driver	mysql2 (Node.js)	Promise-based MySQL client; connection pooling; parameterized queries
Frontend	HTML5, CSS3, Vanilla JavaScript	Single-page dispatcher dashboard; fetch() API for all data calls
Interface Tool	MySQL Workbench	Schema design, query testing, and stored procedure debugging
Version Control	Git / GitHub	Source control and project hosting

11. ACTUAL OUTCOMES

The following outcomes were achieved upon full implementation and testing of the system:

Database Outcomes

- **Fully normalized schema delivered:** All five tables (Driver, Ambulance, Hospital, Emergency_Request, Assignment) were created in MySQL in Third Normal Form with zero partial or transitive dependencies. All primary keys, foreign keys, UNIQUE constraints, NOT NULL constraints, ENUM domains, and the CHECK constraint on available_beds were verified to function correctly.
- **Stored function operational:** The calculate_distance function computes Haversine distances correctly for test coordinate pairs across Punjab (Chandigarh, Patiala, Ludhiana, Mohali) and was verified against known geographic distances.
- **Stored procedure allocates correctly:** The allocate_ambulance procedure successfully selects the geographically nearest AVAILABLE ambulance, assigns the first available hospital, and commits all four resource updates atomically. Under all test conditions, partial allocation states were never observed.
- **ACID compliance verified:** Transaction rollback was tested by artificially triggering exceptions mid-procedure; in all cases the database returned to its pre-transaction state with no partial writes. The before_assignment_insert trigger correctly blocked assignments to hospitals with zero beds in all test cases.
- **Concurrency control demonstrated:** Simultaneous API calls were issued for two emergency requests with the same target ambulance. Row-level locking caused the second transaction to block and then select the next nearest available unit; double-booking did not occur in any test run.
- **Three triggers functioning:** The BEFORE INSERT trigger raised the correct SQLSTATE error on zero-bed hospitals. The AFTER UPDATE trigger correctly freed the ambulance and closed the request status every time a completion_time was set.
- **Three views created and queried:** The available_ambulances, active_assignments, and hospital_capacity views were created and verified to return correct results consistent with the underlying table data.

Application Outcomes

- **REST API fully operational:** All 14 API endpoints respond correctly, including POST /api/requests (which triggers the stored procedure), PUT

/api/ambulances/:id/complete (which sets completion_time and fires the trigger chain), and all GET endpoints for dashboard data.

- **Single-page dashboard functional across all six pages:** Dashboard (live metrics, recent requests, hospital bars, fleet strip), Emergency Requests (new-request form, status filter tabs, manual Allocate button), Ambulances (fleet table, Mark done), Hospitals (bed occupancy cards, Update beds), Assignments (full join table, Mark complete), and Reports (daily summary, bar charts, hourly distribution) were all tested with real data.
- **Auto-refresh working:** The dashboard correctly polls all three data sources every 30 seconds and updates all metric cards, the requests list, hospital bars, and fleet strip without a page reload.
- **Seed data verified:** Eight drivers, eight ambulances, four hospitals, and seven emergency requests were seeded and correctly reflected across all dashboard pages.

Educational Outcomes

- Demonstrated practical application of all six PL/SQL construct types covered in the course: stored procedures, functions, triggers, views, cursors (implicit, within procedure), and exception handling.
- Demonstrated use of SQL SELECT with 5-table INNER JOIN, LEFT JOIN, conditional aggregation (CASE WHEN inside SUM), GROUP BY, HAVING, ORDER BY, and subqueries.
- Demonstrated DDL (CREATE TABLE, CREATE FUNCTION, CREATE PROCEDURE, CREATE TRIGGER, CREATE VIEW), DML (INSERT, UPDATE with parameterized queries), and TCL (START TRANSACTION, COMMIT, ROLLBACK).
- Provided a concrete, working demonstration of how ACID properties and row-level locking solve the real-world double-booking problem — a scenario that cannot be safely handled with file-based systems.